

Using Epistemic Observability in Agentic Systems for Combating Hallucinations

Tony Mason
The University of British Columbia

Vaastav Anand
Max Planck Institute for Software Systems

Abstract

LLMs are increasingly deployed as components of production systems, introducing a fundamental challenge: generated outputs may contain fabrications, while verification incurs computational and monetary cost. Systems must therefore decide how to allocate limited verification budgets: what reliability does each level of verification investment buy?

We argue that text-only observation, even with bounded supervision, is structurally insufficient for reliably assessing the epistemic validity of model outputs. To address this limitation, we introduce epistemic observability, an interface that exposes internal computational telemetry alongside generated text. These signals provide systems with empirical evidence for estimating hallucination risk and prioritizing verification effort. We present a four-tier epistemic observability hierarchy that characterizes the trade-off between verification cost and the amount of epistemic information available to downstream systems. As a proof-of-concept, we design a tensor-generation interface that exposes lightweight epistemic telemetry and demonstrate that tensor-guided judges consistently outperform text-only baselines across all verification budgets, improving accuracy by 3.3–4.6 percentage points on average across four model families.

CCS Concepts: • Computer systems organization → Reliability; • Software and its engineering → Empirical software validation.

Keywords: epistemic observability, agentic reliability

ACM Reference Format:

Tony Mason and Vaastav Anand. 2026. Using Epistemic Observability in Agentic Systems for Combating Hallucinations. In *Proceedings of Practical Adoption Challenges of ML for Systems (PACMI'26)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. *PACMI'26, Prague, Czechia*

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Systems incorporating language model components face a verification problem. The components generate text by sampling from learned probability distributions, and some fraction of that text is fabricated: fluent, confident, and wrong. We use the term *fabrication* throughout this paper to emphasize that the system's default behavior under coherence optimization is reasonable and not a malfunction to be patched. Xu et al. [21] prove that fabrication is inevitable for LLMs over the class of computable functions.

Consequently, every system that deploys a language model component must allocate a budget: how much of the output to verify, which outputs to prioritize, and what signals to use for triage. Verification cost varies by mechanism: a citation lookup costs latency, a second model as judge costs compute, human-in-the-loop validation costs both latency and automation flexibility.

The fundamental source of these costs is the interface through which language models are observed. Today's systems expose only the generated text while hiding the model's internal computation, which contains signals that may distinguish grounded generation from fabrication. Because these signals are discarded before reaching the supervisor, they cannot be recovered from the output alone. Under bounded supervision, grounded and fabricated internal states can be observationally equivalent through the text channel alone, independent of model scale or training procedure [13]. This mismatch between what the model internally observes during generation and what the supervisor can observe afterward creates a fundamental observability gap.

This observability gap has direct consequences for verification cost: verification mechanisms must now compensate for information discarded at the model boundary as a supervisor limited to text spends its budget reconstructing information that was available during generation but hidden by the interface, instead of focusing on the small fraction of outputs that genuinely require scrutiny. This paper therefore asks the systems question: if language models exposed richer observational interfaces, how would that change the cost, effectiveness, and return on investment of verification?

Our key insight is that the information needed to enrich this interface already exists during inference. During inference, language models naturally produce auxiliary information that reflects their internal computation. We refer to these signals as *telemetry*: byproducts of inference that the model does not explicitly choose to communicate. This

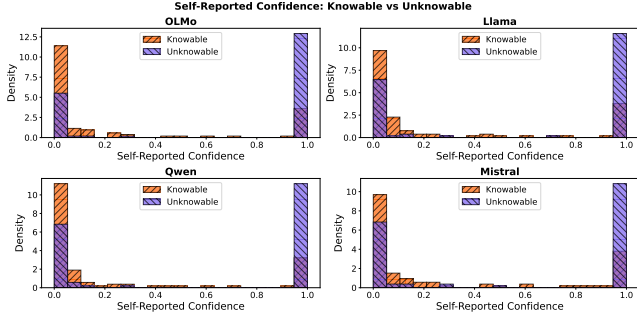


Figure 1. Self-reported confidence is inverted: all four models report higher confidence on unknowable queries

distinguishes telemetry from the model’s *testimony*—the generated text itself. Whereas testimony is optimized to satisfy the prompting objective and can therefore be misleading, telemetry arises as a consequence of computation: under current training objectives the model cannot alter it without altering the computation itself.

We introduce *epistemic observability*: an interface that exposes this telemetry alongside the standard text output. Unlike interpretability, which explains why a model activates particular neurons, or explainability, which generates post-hoc rationales, epistemic observability surfaces byproducts of inference that estimate whether the generated content is grounded in the model’s knowledge or likely fabricated.

To support a range of applications, we propose a multi-tier epistemic observability interface, where each successive tier exposes richer telemetry about the generation process at greater computational cost. A creative brainstorming assistant may require only Tier 0 observability, whereas a medical decision-support system may justify the cost of higher tiers; the hierarchy makes explicit what additional epistemic information each level of investment provides.

As a proof of concept, we build a tensor-generation interface exposing per-token entropy (Tier 1) and attention summaries (a low-cost Tier 2 signal), and show that tensor-guided judges outperform text-only baselines at every verification budget, by 3.3–4.6 percentage points on average across four model families.

2 Background & Motivation

A language model maps input tokens to output tokens, producing internal signals (entropy, attention patterns, logprobabilities) as computational byproducts. The training corpus contains both truth-tracking institutional text and misinformation. Probing how a model trained on this mixture responds yields four canonical query types that span the verification space: *adversarial truths* (true but implausible), *shattered lies* (no training-data grounding), *deceived lies* (fabrications that pattern-match plausible language), and *confused truths* (counterintuitive facts).

A model trained on this corpus inherits its mixed regularities: base models fabricate substantially on factual benchmarks, yet also encode internal representations of correctness that can be elicited under the right conditions [3]. Consequently, the model is neither reliably honest nor uniformly deceptive. This uncertainty makes verification necessary: the challenge is determining which outputs require scrutiny and at what cost.

2.1 Testimony & Telemetry

Architecture primer. A transformer generates text one token at a time. At each step it produces a probability distribution over its vocabulary, selects a token, and conditions the next step on all prior output. Each step passes through a stack of layers producing *attention patterns* (weighted relevance scores between positions) and intermediate representations to generate a final text output, which we call its *testimony*. The *entropy* of the output distribution, the attention patterns, and the *log-probabilities* of selected tokens are byproducts of this computation, which we call its *telemetry*.

From a verification cost perspective, testimony is cheap to produce and inspect but unreliable; telemetry is more expensive to export and process but harder to game (under current training regimes (§3.2)), because it is a byproduct of the computation rather than a product of the optimization objective. Existing models expose only its testimony and discard its telemetry.

2.2 The Text Channel Cannot Self-Verify

To demonstrate the inadequacy of text-only verification, we use a taxonomy of query types defined by construction: *knowable* queries have a determinate, publicly verifiable answer (e.g., national capitals, well-documented scientific facts), whereas *unknowable* queries have no correct answer for any model, because the entity does not exist, the event has not happened, or the information is private. “Knowable” is a property of the query, not a claim about a particular model’s weights; a model may still fail a knowable query. We report each model’s self-confidence score for the self-reported confidence baseline, the cheapest verification strategy available to a bounded supervisor. In this baseline, the supervisor asks the model “How confident are you?” after generation and tests whether the model can introspect and report its epistemic state through text.

Figure 1 shows that self-reported confidence is *inverted*: all four models report higher confidence on unknowable queries than on knowable ones, yielding AUC 0.28–0.36 across architectures (all below random). A user who asks “how confident are you?” receives a number that anticorrelates with epistemic grounding, and an adversary could exploit the inversion to make fabrications appear maximally confident. Under bounded oversight, a text-only supervisor cannot distinguish truthful uncertainty from confident fabrication.

3 Epistemic Observability

We use *epistemic state* to describe the model’s internal computational state as it relates to the reliability of its generation. This state provides evidence about whether the model is generating from information it reliably represents or constructing a plausible response without such support.

We define *epistemic observability* as the ability of a language model to expose telemetry about this state during inference—decoding statistics, internal representations, computational traces—so that downstream components of the system can use this otherwise hidden evidence to assess fabrication risk of a generated output.

3.1 Observability Tiers

We present a four-tier epistemic observability hierarchy that characterizes the trade-off between verification cost and the amount of epistemic information available to downstream systems for assessing reliability of the generated text.

Tier 0: Output Observability. Tier 0 exposes only the generated text, treating the model as a black box. This is the interface of most commercial APIs and the basis of existing validation techniques: retrieval-based verification, secondary LLM judges, symbolic validators, and application-specific consistency checks. With no internal telemetry, verification must rely entirely on properties of the output, often at the cost of expensive external computation.

Tier 1: Confidence Observability. Tier 1 augments the generated text with lightweight telemetry produced during decoding such as token probabilities, entropy [10, 20], confidence scores, and other decoding statistics extractable with negligible overhead. These give a coarse estimate of the model’s uncertainty [17] and identify portions of the output that may warrant validation. They are often imperfectly calibrated and may fail to distinguish confidently stated hallucinations from grounded generations.

Tier 2: Representation Observability. Tier 2 exposes information derived from the model’s internal representations. Systems may access hidden-state embeddings, layer activations, attention statistics, or learned probes [1, 2] that estimate properties of the latent epistemic state. Simple probes can catch deviant model behaviors [4, 5, 12], and semantic entropy probes [7, 8] and hidden-state classifiers [22] show that these representations carry substantially richer information about reliability than logits alone, at a cost still well below exhaustive external validation.

Tier 3: Mechanistic Observability. Tier 3 exposes telemetry obtained through mechanistic analysis of the model’s computation. Mechanistic techniques trace information flow through neurons, attention heads, and latent features [18]. Tier 3 telemetry can distinguish between failure modes—a response with no stable internal support versus one produced by a confident but spurious pathway—rather than only flagging that a failure occurred. We do not claim that Tier 3

identifies the *cause* of a hallucination in the interventional sense (a component which, when altered, changes the outcome); that is the aim of mechanistic interpretability, whose tools Tier 3 exposes but does not itself deliver.

3.2 Telemetry is Coupled to Computation

Telemetry is produced by the same model that produces the testimony, so it is mutable. Fine-tuning can shift it, and a model trained against a monitored signal could learn to manipulate it deliberately. Changing the telemetry such as the entropy trace or attention geometry requires changing the computation that produces the text. In contrast, behavioral signals (response length, hedging) can be adapted by training without changing the underlying computation. Thus, telemetry has an additional robustness property as compared to the behavioral signal that the telemetry is not *separately* controllable from the computation being monitored.

Under current training objectives, models receive no explicit incentive to make such changes for the purpose of evading a telemetry-based monitor. Independent work on persona monitoring supports the asymmetry: activation projections reveal distinct internal configurations for different epistemic conditions even when the text is similar [11].

This distinction comes with two important limits. First, the property holds under *current* pipelines: the models we evaluate are instruction-tuned, so standard post-training does not erase the signal in our data. Optimization that explicitly incorporates the monitor into the training objective, however, constitutes a different regime about which we make no claim. Second, mechanistic tools can already locate the subspaces such an attack would target [19]. Exposing telemetry therefore makes such manipulation observable and auditable rather than hidden behind the model interface.

3.3 Runtime Integration

Epistemic observability is a triage primitive: a system uses the observability information to estimate fabrication risk and expend its verification budget accordingly. This separates *observation* from *action*: the epistemic observability interface exposes evidence about the model’s epistemic state, while a deployment-specific policy determines what to do with that evidence. Possible actions form a ladder of increasing cost, including *abstain or flag* (withhold or annotate the output); *re-generate* (sample again or add reasoning steps and require agreement); *tool verification* (route the output to a bounded external check—a citation lookup, a database query, a test); *escalation* (request telemetry from a higher observability tier); and *human review*.

The observability interface integrates with an agent runtime at three points. At the *serving layer*, telemetry is exported alongside the generated text, with the amount and cost of telemetry determined by the selected observability tier. At the *composition boundary*, where the runtime intercepts tool and sub-agent calls, the telemetry can gate

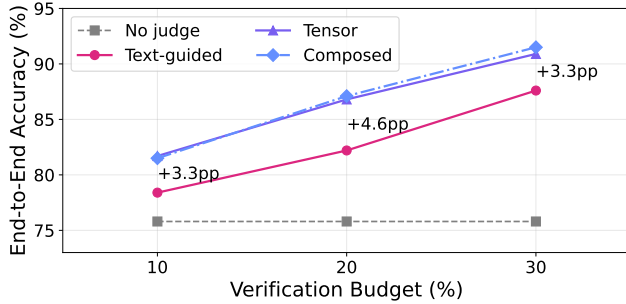


Figure 3. End-to-end accuracy vs. verification budget for four judge conditions, averaged across four architectures.

investment strategy to its return: per-token entropy discriminates among outputs of similar length, reaching cases the text channel cannot. Concurrent work scales verification on the judge side by taking the expectation over a judge’s scoring-token distribution rather than its discrete score [9]; our signals instead come from the generator’s own computation, which—unlike a judge’s scores—the model does not separately control (§3.2). Per-model results confirm the pattern is consistent across all four tested model families. At 10% budget, tensor-guided verification improves accuracy over the unverified baseline for every model tested: OLMo (69.5% → 75.1%), Llama (82.5% → 88.1%), Qwen (87.0% → 91.7%), and Mistral (64.0% → 72.0%). The signal is not confined to small local models: using only the top-5 log-probabilities that serverless APIs return (a lower bound on full-vocabulary entropy), mean entropy separates knowable from unknowable queries on five further architectures from 4B to 235B parameters, including two mixture-of-experts models, with AUC 0.65–0.88 [13].

Composed judges and complementary failure modes. At 30% budget, the composed judge (91.5%) outperforms the tensor-guided judge alone (90.9%). The advantage is small in aggregate but structurally significant: it comes entirely from citation queries where entropy signals *invert*. Fabricated citations produce lower entropy than real ones because the model generates plausible fakes more fluently than it retrieves specific bibliographic details.

5 Conclusion

Systems built on language model components must decide how much verification to buy and where to spend it. We introduce epistemic observability—a hierarchy of tiers, each exposing richer epistemic information at greater cost—to provide the cost surface for that decision, and build a low-tier tensor interface that reduces the verification budget needed to curb hallucinations.

Acknowledgments

We thank the anonymous PACMI’26 reviewers for feedback that substantially improved this paper.

References

- [1] Guillaume Alain and Yoshua Bengio. 2016. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644* (2016).
- [2] Yonatan Belinkov. 2022. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics* 48, 1 (2022), 207–219.
- [3] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. 2023. Discovering Latent Knowledge in Language Models Without Supervision. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=ETKGuby0hcs>
- [4] Nicholas Goldowsky-Dill, Bilal Chughtai, Stefan Heimersheim, and Marius Hobbhahn. 2025. Detecting strategic deception using linear probes. *arXiv preprint arXiv:2502.03407* (2025).
- [5] Jiatong Han, Neil Band, Muhammed Razzak, Jannik Kossen, Tim GJ Rudner, and Yarin Gal. 2025. Simple factuality probes detect hallucinations in long-form natural language generation. *Findings of the Association for Computational Linguistics: EMNLP* (2025), 16209–16226.
- [6] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Giana Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv:2310.06825* [cs.CL] <https://arxiv.org/abs/2310.06825>
- [7] Jannik Kossen, Jiatong Han, Muhammed Razzak, Lisa Schut, Shreshth Malik, and Yarin Gal. 2024. Semantic entropy probes: Robust and cheap hallucination detection in llms. *arXiv preprint arXiv:2406.15927* (2024).
- [8] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. *arXiv:2302.09664* [cs.CL] <https://arxiv.org/abs/2302.09664>
- [9] Jacky Kwok, Shulu Li, Pranav Atreya, Yuejiang Liu, Yixing Jiang, Chelsea Finn, Marco Pavone, Ion Stoica, and Azalia Mirhoseini. 2026. LLM-as-a-Verifier: A General-Purpose Verification Framework. *arXiv:2607.05391* [cs.AI] <https://arxiv.org/abs/2607.05391>
- [10] Chen Ling, Xujiang Zhao, Xuchao Zhang, Wei Cheng, Yanchi Liu, Yiyu Sun, Mika Oishi, Takao Osaki, Katsushi Matsuda, Jie Ji, et al. 2024. Uncertainty Quantification for In-Context Learning of Large Language Models. *arXiv preprint arXiv:2402.10189* (2024).
- [11] Christina Lu, Jack Gallagher, Jonathan Michala, Kyle Fish, and Jack Lindsey. 2026. The Assistant Axis: Situating and Stabilizing the Default Persona of Language Models. *arXiv:2601.10387* [cs.CL] <https://arxiv.org/abs/2601.10387>
- [12] Monte MacDiarmid, Timothy Maxwell, Nicholas Schiefer, Jesse Mu, Jared Kaplan, David Duvenaud, Sam Bowman, Alex Tamkin, Ethan Perez, Mrinank Sharma, Carson Denison, and Evan Hubinger. 2024. Simple probes can catch sleeper agents. <https://www.anthropic.com/news/probes-catch-sleeper-agents>
- [13] Tony Mason and Vaastav Anand. 2026. Epistemic Observability in Language Models. *arXiv:2603.20531* <https://arxiv.org/abs/2603.20531>
- [14] Meta AI. 2024. The Llama 3 Herd of Models. *arXiv:2407.21783* [cs.AI] <https://arxiv.org/abs/2407.21783>
- [15] Team Olmo, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee Chen, Michael Noukhovitch, Nathan Lambert, Pete Walsh, Pradeep Dasigi, Robert Berry, Saumya Malik, Saurabh Shah, Scott Geng, Shane Arora, Shashank Gupta, Taira Anderson, Teng Xiao, Tyler Murray, Tyler Romero, Victoria Graf, Akari Asai, Akshita Bhagia, Alexander Wettig, Alisa Liu, Aman Rangapur, Chloe Anastasiades, Costa Huang, Dustin Schwenk, Harsh Trivedi, Ian Magnusson, Jaron Lochner, Jiacheng Liu, Lester James V. Miranda, Maarten Sap, Malia Morgan, Michael Schmitz, Michal Guerquin, Michael Wilson, Regan Huff, Ronan Le Bras, Rui Xin, Rulin Shao, Sam Skjonsberg,

- Shannon Zejiang Shen, Shuyue Stella Li, Tucker Wilde, Valentina Pyatkin, Will Merrill, Yapei Chang, Yuling Gu, Zhiyuan Zeng, Ashish Sabharwal, Luke Zettlemoyer, Pang Wei Koh, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2025. Olmo 3. arXiv:2512.13961 [cs.CL] <https://arxiv.org/abs/2512.13961>
- [16] Qwen Team. 2025. Qwen3 Technical Report. Alibaba Cloud technical report. <https://qwenlm.github.io/blog/qwen3/>
- [17] Ola Shorinwa, Zhiting Mei, Justin Lidard, Allen Z Ren, and Anirudha Majumdar. 2025. A survey on uncertainty quantification of large language models: Taxonomy, open research challenges, and future directions. *Comput. Surveys* 58, 3 (2025), 1–38.
- [18] Shriyank Somvanshi, Md Monzurul Islam, Amir Rafe, Anannya Ghosh Tusti, Arka Chakraborty, Anika Baitullah, Tausif Islam Chowdhury, Nawaf Alnawmasi, Anandi Dutta, and Subasish Das. 2026. Bridging the black box: a survey on mechanistic interpretability in AI. *Comput. Surveys* 58, 8 (2026), 1–35.
- [19] Thomas Winninger, Boussad Addad, and Katarzyna Kapusta. 2025. Using Mechanistic Interpretability to Craft Adversarial Attacks against Large Language Models. arXiv:2503.06269 [cs.LG] <https://arxiv.org/abs/2503.06269>
- [20] Yijun Xiao and William Yang Wang. 2021. On hallucination and predictive uncertainty in conditional language generation. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. 2734–2744.
- [21] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. 2024. Hallucination is Inevitable: An Innate Limitation of Large Language Models. arXiv:2401.11817 [cs.CL]
- [22] Zhenliang Zhang, Xinyu Hu, Huixuan Zhang, Junzhe Zhang, and Xiaojun Wan. 2025. ICR probe: Tracking hidden state dynamics for reliable hallucination detection in LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 17986–18002.